Function Overloading:

Function overloading is a **C++ programming** feature that allows us to have more than one function having same name but different parameter list or signature , when I say parameter list, it means the data type , sequence of the parameters and no of parameters , for example the parameters list of a function `myfuncn(int a, float b)` is `(int, float)` which is different from the function `myfuncn(float a, int b)` parameter list `(float, int)`. Function overloading is a **compile-time polymorphism**.

Few more examples for function overloading:

```
sum(int num1, int num2)
sum(int num1, int num2, int num3)
sum(int num1, double num2)
```

All of the above three cases are valid case of overloading. We can have any number of functions, just remember that the parameter list should be different. For example:

```
int sum(int, int)
double sum(int, int)
```

This is not allowed as the parameter list is same. Even though they have different return types, its not valid.

Functions having different return types and same parameter list cannot be overloaded. However if the functions have different parameter list then they can have same or different return types to be eligible for overloading.

In short the return type of a function does not play any role in function overloading.

Function overloading is simply based on type of input parameters,order of the parameters and no of parameters.

Example Program for function overloading:

```
#include <iostream>
using namespace std;
int sum(int num1,int num2) {
      return num1+num2;
 }
int sum(int num1,int num2, int num3) {
 return num1+num2+num3;
 }

int main(void) {
           cout<<sum(20, 15)<<endl;
           cout<<sum(81, 100, 10);
          return 0;
}
```

Vector India